

# Senior Android Technical Assessment

## Mini Task: AIDL + TCP + Room + RxJava + Dagger 2 + Foreground Service

Implementation language: Java only

Android Java ecosystem, Dagger 2, RxJava, Room, and Android platform services

### Purpose

This is a compact technical task for evaluating a senior Android developer. It focuses on production-grade thinking: IPC, TCP networking, local persistence, dependency injection, foreground execution, thread-safety, and performance awareness.

## 1. Task Overview

Build two small Android applications:

- Merchant App: sends a payment-like request and receives callbacks.
- Payment Core App: receives the request through AIDL, stores it in Room, processes it through a TCP client, updates the database, and returns the result through AIDL callback.

This assessment is intentionally smaller than a full production payment system, but it must still be implemented with senior-level engineering discipline.

**Positive evaluation note:** This is a technical task. Any creative improvement that increases performance, speed, reliability, maintainability, observability, thread-safety, resource efficiency, or startup/response time will be considered a positive evaluation signal. The candidate should briefly explain the reasoning behind each improvement.

## 2. Technology Constraints

| Area                 | Requirement                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Language             | Java only. Kotlin is not allowed for the core implementation.                                                                           |
| Dependency Injection | Dagger 2 only. Hilt is not allowed.                                                                                                     |
| Async / Reactive     | RxJava 2 or RxJava 3. Coroutines are not allowed.                                                                                       |
| Persistence          | Room with DAO and Entity classes written for Java usage.                                                                                |
| IPC                  | AIDL with request and callback interfaces.                                                                                              |
| Networking           | Raw TCP Socket client with timeout handling.                                                                                            |
| Background Execution | Foreground Service with Notification.                                                                                                   |
| Architecture         | Clean Architecture with clear presentation, domain, and data boundaries.                                                                |
| Threading            | Single active transaction guard using AtomicBoolean, ReentrantLock, synchronized, SingleThreadExecutor, or serialized RxJava Scheduler. |

## 3. Scenario and Core Flow

The Merchant App sends a transaction request to the Payment Core App. The Payment Core App must process the request and send status updates and the final result back to the Merchant App.

1. Receive the request through AIDL.
2. Reject the request if another transaction is already active.
3. Validate the request.
4. Persist the transaction in Room.
5. Start a Foreground Service and show a Notification.
6. Connect to a mock TCP server.
7. Send the request and wait for the response.
8. Parse and validate the response.
9. Update Room with success or failure.
10. Send the final result through AIDL callback.
11. Stop the service if keepServiceAlive is false.
12. Keep the service active but release the transaction lock if keepServiceAlive is true.

## 4. AIDL APIs

The Payment Core App must expose these APIs through AIDL:

```
void startTransaction(in PaymentRequest request, IPaymentCallback callback);
PaymentResult getTransactionStatus(String requestId);
```

The callback interface must support:

```
void onTransactionStarted(String requestId);
void onTransactionProgress(String requestId, String status);
void onTransactionCompleted(in PaymentResult result);
void onTransactionFailed(in PaymentResult result);
```

## 5. PaymentRequest Model

| Field            | Java Type | Rule                                        |
|------------------|-----------|---------------------------------------------|
| requestId        | String    | Required, unique, not blank                 |
| amount           | long      | Must be greater than 0                      |
| terminalId       | String    | Required, not blank                         |
| traceNumber      | long      | Must be greater than 0                      |
| keepServiceAlive | boolean   | Controls service lifecycle after completion |

## 6. PaymentResult Model

| Field        | Java Type                   |
|--------------|-----------------------------|
| requestId    | String                      |
| status       | String or enum-mapped value |
| responseCode | String, nullable            |
| rrn          | String, nullable            |
| message      | String, nullable            |
| durationMs   | long                        |

## 7. Transaction States

The candidate must model transaction state clearly. Scattered string literals inside random classes are not acceptable.

```
RECEIVED
STORED
PROCESSING
CONNECTING
SENDING
WAITING_RESPONSE
SUCCESS
FAILED
CANCELLED
```

The implementation should prevent an already-final transaction from being completed or failed again.

## 8. Room Requirements

Create a Room table named transactions.

| Field        | Notes               |
|--------------|---------------------|
| id           | Primary key         |
| requestId    | Unique and indexed  |
| amount       | Transaction amount  |
| terminalId   | Terminal identifier |
| traceNumber  | Trace number        |
| status       | Current state       |
| responseCode | Nullable            |
| rrn          | Nullable            |
| message      | Nullable            |
| createdAt    | Timestamp           |

|                  |                        |
|------------------|------------------------|
| updatedAt        | Timestamp              |
| keepServiceAlive | Service lifecycle flag |

Required DAO operations:

```
insertTransaction(...)
getTransactionByRequestId(...)
updateTransactionStatus(...)
markTransactionSuccess(...)
markTransactionFailed(...)
```

- requestId must be unique.
- Hint: keep state updates consistent when a transaction moves to a final status.

## 9. TCP Requirements

The Payment Core App must connect to a mock TCP server. The message format can be simple JSON, but the client must be deterministic and reliable.

Example request:

```
{
  "requestId": "REQ-1001",
  "amount": 250000,
  "terminalId": "T123",
  "traceNumber": 10001
}
```

Example response:

```
{
  "requestId": "REQ-1001",
  "responseCode": "00",
  "rrn": "123456789012",
  "message": "Approved"
}
```

Minimum TCP behavior:

- Connect timeout.
- Read timeout.
- Safe write handling.
- Response parsing.
- Invalid response handling.
- Safe socket closing in success and failure paths.

## 10. RxJava Pipeline

The transaction pipeline must be implemented with RxJava.

```
validate request
-> insert into Room
-> start foreground service
-> connect TCP
-> send request
-> receive response
-> parse response
-> update Room
```

- > send callback
- > stop or keep service

Hint: use a SchedulerProvider for database work, TCP work, and callback delivery.

## 11. Dagger 2 Requirements

Use Dagger 2 to provide the core dependencies listed below.

| Dependency              | Expected Injection Target        |
|-------------------------|----------------------------------|
| Room Database           | Repository / DAO provider        |
| TransactionDao          | Repository                       |
| PaymentRepository       | UseCase                          |
| StartTransactionUseCase | AIDL Service or orchestrator     |
| TcpClient               | Repository or remote data source |
| SchedulerProvider       | UseCase / Repository             |
| NotificationHelper      | Foreground Service controller    |

## 12. Thread-Safety and Foreground Service Requirements

### Thread-safety

Only one active transaction is allowed at a time. The implementation must protect against concurrent calls and race conditions.

- Two simultaneous startTransaction calls must not both enter processing.
- Duplicate requestId must be rejected or resolved safely.
- A final transaction must not be updated again.
- The active transaction lock must always be released in success, failure, and cancellation paths.
- The service lifecycle must not conflict with callback delivery.

Acceptable mechanisms include AtomicBoolean, ReentrantLock, synchronized blocks, SingleThreadExecutor, or a serialized RxJava Scheduler. The candidate must explain why the selected mechanism is safe.

### Foreground Service

The Payment Core App must start a Foreground Service during active transaction processing and show a Notification.

| Notification State     | When It Should Appear           |
|------------------------|---------------------------------|
| Processing transaction | After request validation starts |
| Connecting to server   | Before TCP connection           |
| Waiting for response   | After request is sent           |
| Transaction completed  | After successful response       |
| Transaction failed     | After failure or timeout        |

### Auto-stop mode

If keepServiceAlive is false, the service must update the notification, send the final callback, stop foreground mode, and stop itself after the transaction reaches a final state.

### Keep-alive mode

If keepServiceAlive is true, the service may remain active after the transaction. However, the transaction lock must be released, resources must be cleaned up, and the service must not leak callbacks or sockets.